

LLM USAGE LOG

INTERACTION 1: Brainstorming for Architectural Differences

Model: Google Gemini 1.5 Pro

Prompt:

I am comparing a modern transformer model (like Qwen3 or Gemma3) to the original GPT-2 architecture for a class project. Can you help me brainstorm a list of common architectural changes or improvements I should look for in modern transformers that were introduced after GPT-2? I'm looking for high-level concepts like changes in normalization, attention mechanisms, or activation functions.

How I Verified the Output: The model suggested looking for things like Rotary Positional Embeddings (RoPE), SwiGLU/GeGLU activations, LayerNorm variants (like RMSNorm), and Grouped-Query Attention (GQA). I treated this list as a starting point for my research.

1. I searched for each term (e.g., "RMSNorm") in the technical report for my chosen model, Qwen3.
 2. I confirmed that the Qwen3 paper explicitly mentioned using SwiGLU, RoPE, and RMSNorm. This verified that the model's suggestions were relevant to my specific task.
 3. I cross-referenced these concepts with the suggested reading on GPT-2 to confirm they were indeed architectural differences.
-

INTERACTION 2: Code Scaffolding for Parameter Counting

Model: Google Gemini 1.5 Pro

Prompt:

I need to count the parameters of a Hugging Face transformer model in PyTorch. Can you provide a simple Python code snippet that loads a pretrained model, like 'gpt2', and then iterates through all its named parameters, printing the name of each parameter and its total number of elements? Please don't write a full script, just the core loop structure.

How I Verified the Output: The model provided a concise code snippet that imported `GPT2LMHeadModel` from `transformers` and used a `for` loop like `for name, param in model.named_parameters():`.

1. I first ran the code on a very small, simple model I created myself (a single linear layer) to ensure the logic of `param.numel()` was counting correctly.
 2. I then ran the provided scaffolding on the 'gpt2' model as intended.
 3. To double-check its accuracy, I manually calculated the parameters for a single component (the token embedding layer: `wte`) using the formula $V * E$. The number from my manual calculation matched the output of the script for that specific layer, giving me confidence that the script was working correctly for the entire model.
-

INTERACTION 3: Writing Assistance for Clarity

Model: Google Gemini 1.5 Pro

Prompt:

Can you suggest a more academic way to phrase this sentence? "The big deal with RMSNorm is that it's way simpler than LayerNorm because it gets rid of the re-centering step, which makes it run faster and use less memory."

How I Verified the Output: The model provided several alternatives. One suggestion was: "RMSNorm offers a significant computational advantage over traditional LayerNorm by omitting the mean-centering operation. This simplification results in improved efficiency, reducing both latency and memory usage during training and inference."

1. I checked to ensure the core meaning of my original sentence was preserved. The suggested phrasing accurately captured the key points: omitting the "re-centering" (mean) step and the resulting efficiency gains.
 2. I reviewed the original paper on RMSNorm (which I found during my research for Problem 2) to confirm that terms like "computational advantage" and "omitting the mean-centering operation" were technically accurate descriptions.
 3. I chose the phrasing that best fit the tone of my report and integrated it into my own writing.
-

INTERACTION 4: LaTeX Formatting Assistance

Model: Google Gemini 1.5 Pro

Prompt:

I need to write the symbolic formulas for GPT-2 parameter counts in LaTeX. Can you show me how to format these two formulas?

1. For the MLP component: $L * (E * 4E + 4E)$

2. For the self-attention component: $L * (3 * E * E + 3 * E)$ I want them to look professional for my report.

How I Verified the Output: The model provided LaTeX code, such as $L \times (4E^2 + 4E)$ for the MLP and $L \times (3E^2 + 3E)$ for the attention block.

1. I pasted the suggested LaTeX code directly into my document editor and compiled the PDF.
 2. I confirmed that the formulas rendered correctly and looked mathematically clean.
 3. I cross-referenced the output with the style of formulas presented in the assigned reading, "Counting Parameters in Transformers" by Michael Wornow, to ensure my notation was consistent with the source material.
-

INTERACTION 5: Code Debugging and Conceptual Check

Model: Google Gemini 1.5 Pro

Prompt:

My programmatic parameter count for GPT-2 is slightly higher than my manual formula-based count. My formula for embeddings is $V * E + P * E$. My code iterates through `model.named_parameters()`. Is there a reason these might not match exactly in the standard Hugging Face implementation of GPT-2?

How I Verified the Output: The model suggested that the mismatch could be due to tied weights, where the word token embedding weights (`wte`) and the final language model head weights are the same parameter tensor, meaning my manual formula was double-counting them.

1. I inspected the list of `named_parameters` from my Python script and observed there was no separate parameter for the language model head, which supported the model's suggestion.
2. I consulted the official Hugging Face documentation for their GPT-2 model. The documentation confirmed that the weights between the token embeddings and the final linear layer are tied.
3. I adjusted my manual formula to count the shared embedding/decoding weights only once. After this correction, my manual count precisely matched the programmatic count, verifying the model's explanation.